

HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Distributed Computing

Michael Kosyakov
Associate Professor

Denis Tarakanov
Assistant Lecturer

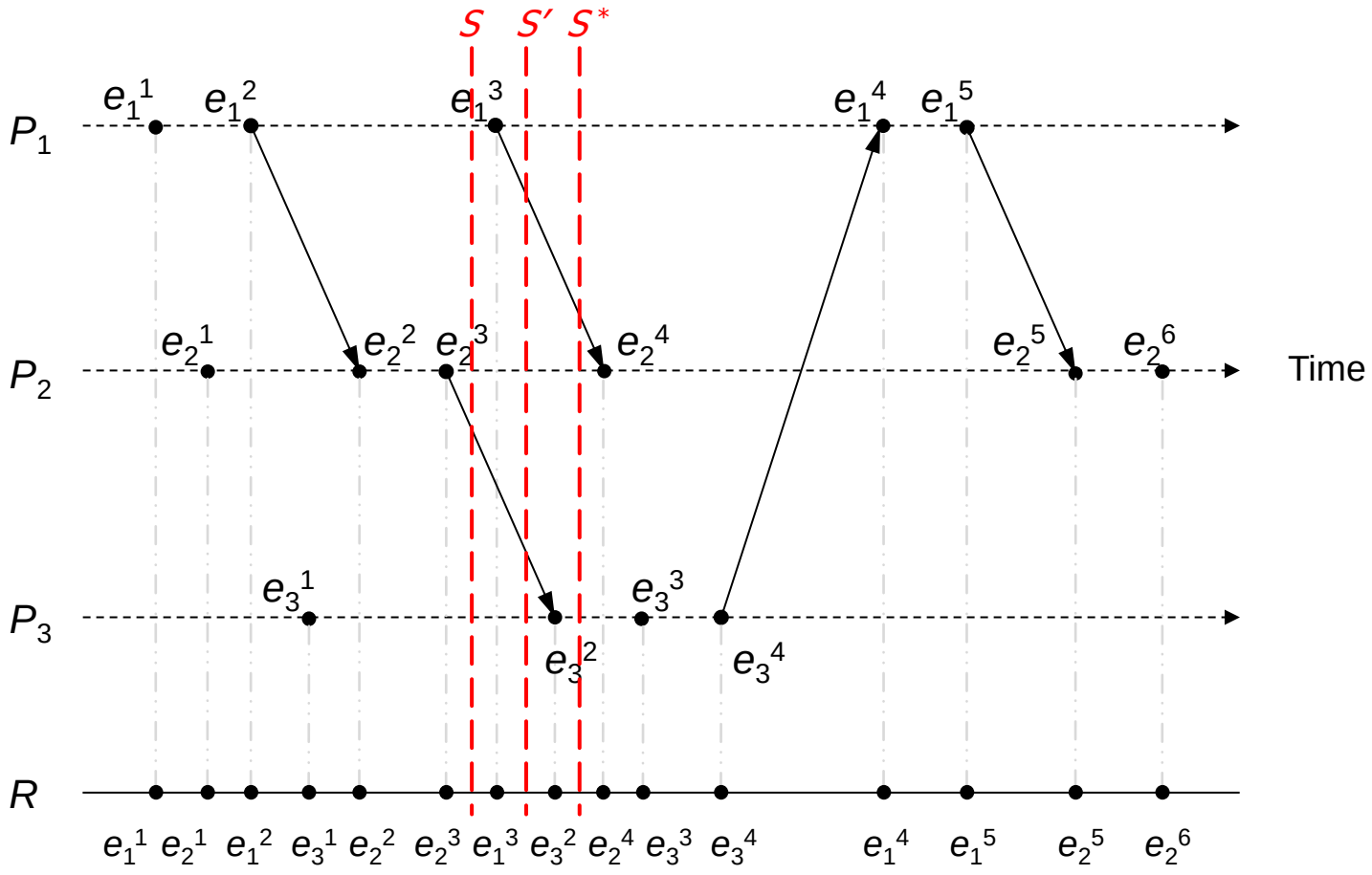
ifmo.distributedclass.bot@gmail.com

Causal order of events

Example of distributed execution

HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Let's recall what distributed execution is …



Independent events



- Each execution $R = (e^0, e^1, \ldots e^x, e^{x+1}, \ldots)$ defines a *unique* sequence of events and associated sequence of global states $(S^0, S^1, \ldots S^x, S^{x+1}, \ldots)$ where event e^x *can occur* in global state S^x and $S^{x+1} = e^x(S^x)$ for each $x \geq 0$
- Each event e^x affects only *part* of the global state of the distributed system, i.e. global state $e^x(S^x)$ immediately after the occurrence of event e^x differs from S only in the state of process P, where e^x occurs, and (possibly) the state of the channel C incident to P
- This leads to the observation that two consecutive events, affecting disjoint parts of the global state, are *independent* and can also occur in reversed order

Independent events



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- To be more precise, let S be a global state, and let $e_i = (P_i, s_i, s_i', m_i, C_i)$ and $e_j = (P_j, s_j, s_j', m_j, C_j)$ be events of *different* processes P_i и P_j, which *both* can occur in S
- Then e_i can happen in global state $S_j = e_j(S)$, and e_j can happen in global state $S_i = e_i(S)$ and $e_i\ (e_j(S)) = e_j\ (e_i(S))$
- It is important to note that e_i and e_j can't be events of sending / receiving the same message m, i.e. m_i and m_j are *different* messages (if any)
  - Otherwise, e_i and e_j can't both occur in S
- Global state $S_i = e_i(S)$ differs from S only in the state of P_i and (possibly, depending on event type) the state of channel C_i

Independent events

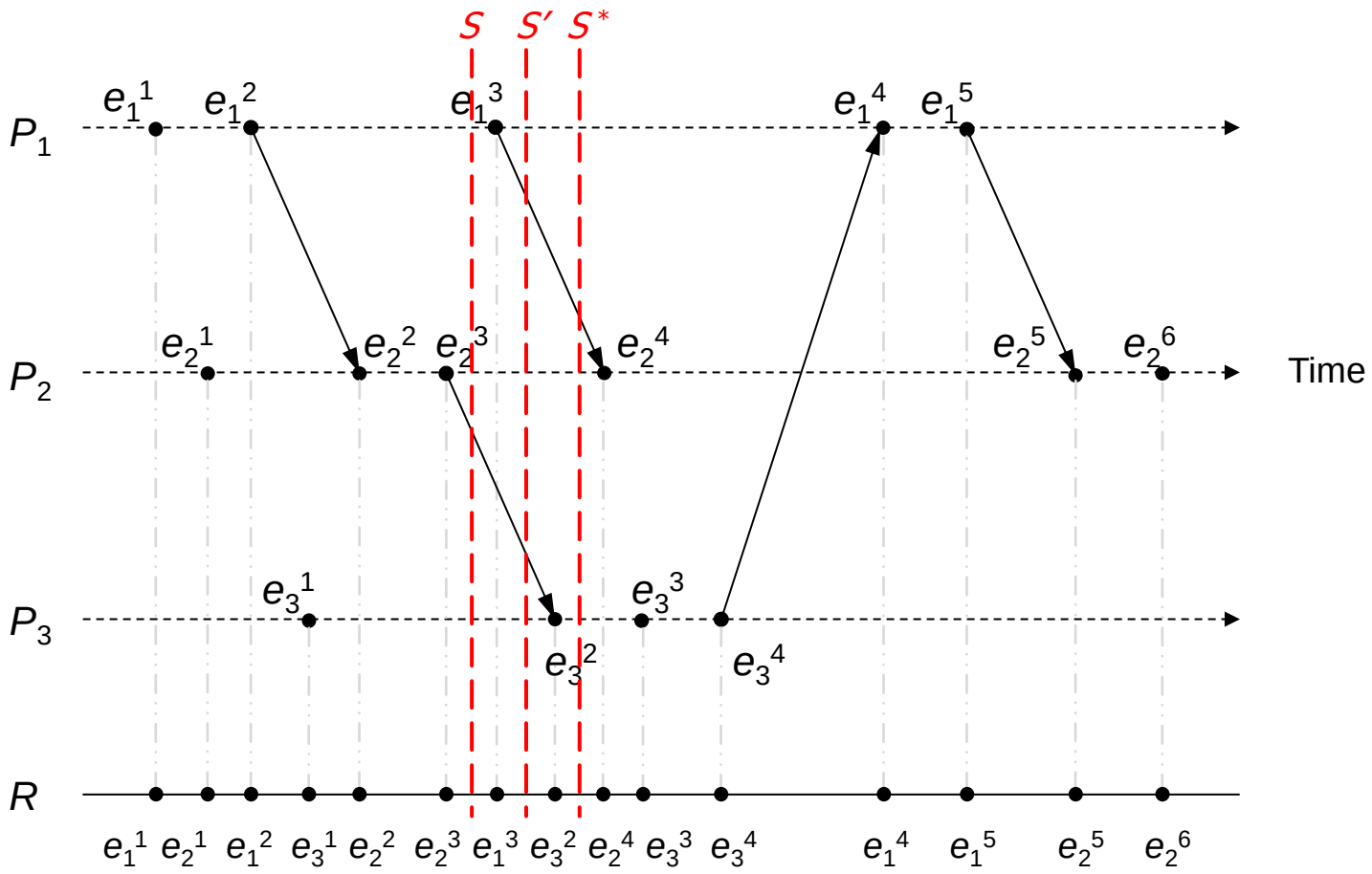


- It is important to note, that the state of P_j is the same in *both* global states S and $S_i = e_i(S)$
- Since e_j doesn't depend on message m_i sent / received on e_i (if any) it means that e_j *can occur* in $S_i = e_i(S)$
  - By a symmetric argument it can be shown that event e_i can occur in global state $S_j = e_j(S)$
- Let's have a look at global state $S_{ij} = e_j(S_i)$: Processes P_i and P_j are in the states s_i' и s_j', messages m_i and m_j are added or removed from their respective channels
- The same situation corresponds to the global state $S_{ji} = e_i(S_j)$, which means that $S_{ij} = S_{ji}$

Independent events

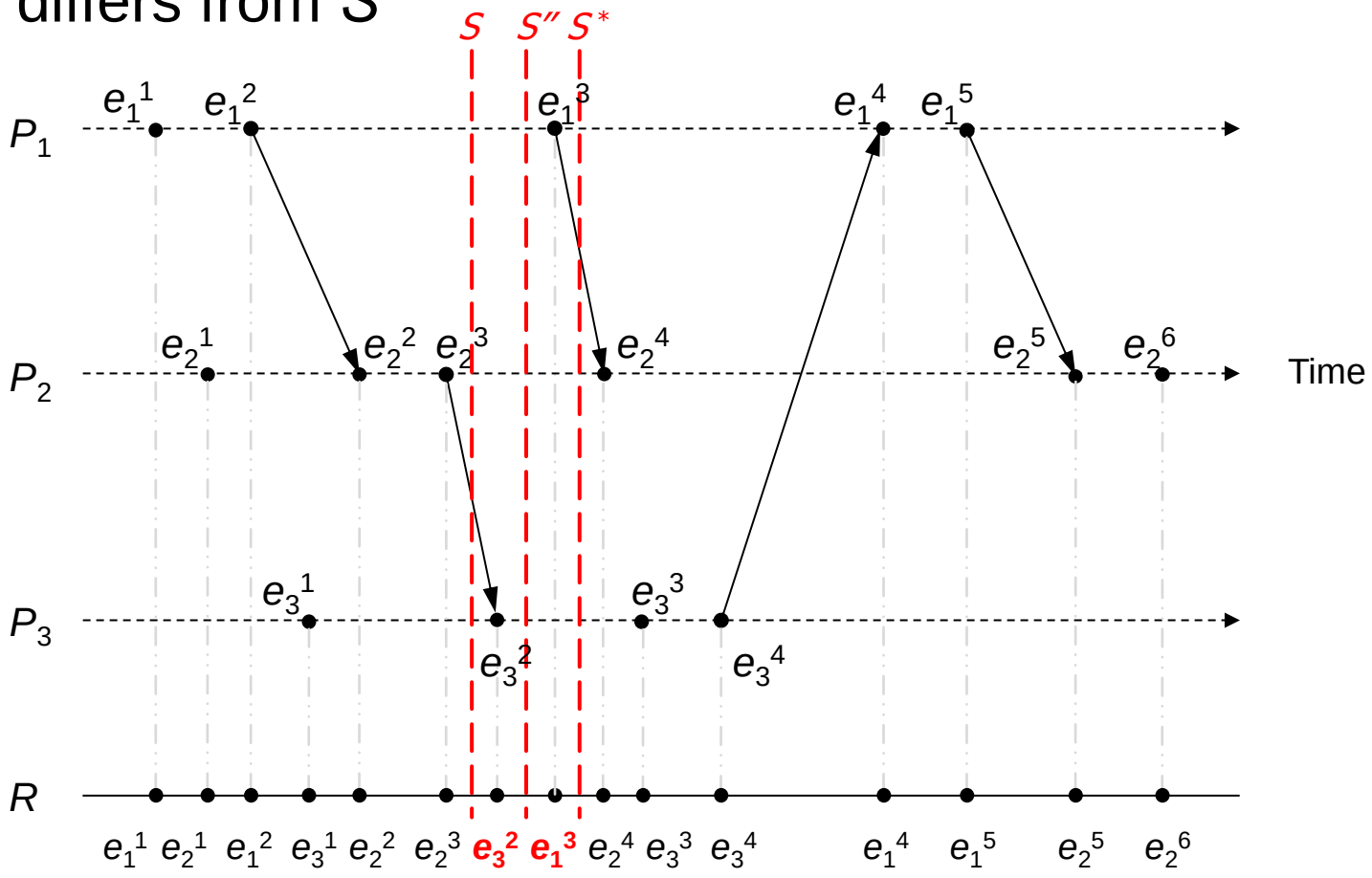
HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Let's reorder events e_1^3 and e_3^2 …



Independent events

… the resulting global state *S** is the same, but *S”* differs from *S’*



Causal relation

- Event can't occur in reversed order only in following two cases:
  - $i = j$, i.e. e_i and e_j are events of the same process
  - e_i is a send event and e_j is the corresponding receive event of the same message
- Otherwise, they can occur in reversed order and yet result in the same global state …
  - … but the system will go through *different* global states, i.e. transitions $S \rightarrow S_i \rightarrow S_{ij}$ are different from transitions $S \rightarrow S_j \rightarrow S_{ji}$ since $S_i \neq S_j$
- We say that there is a (potential) *causal relation* between two events if they can't occur in reversed order

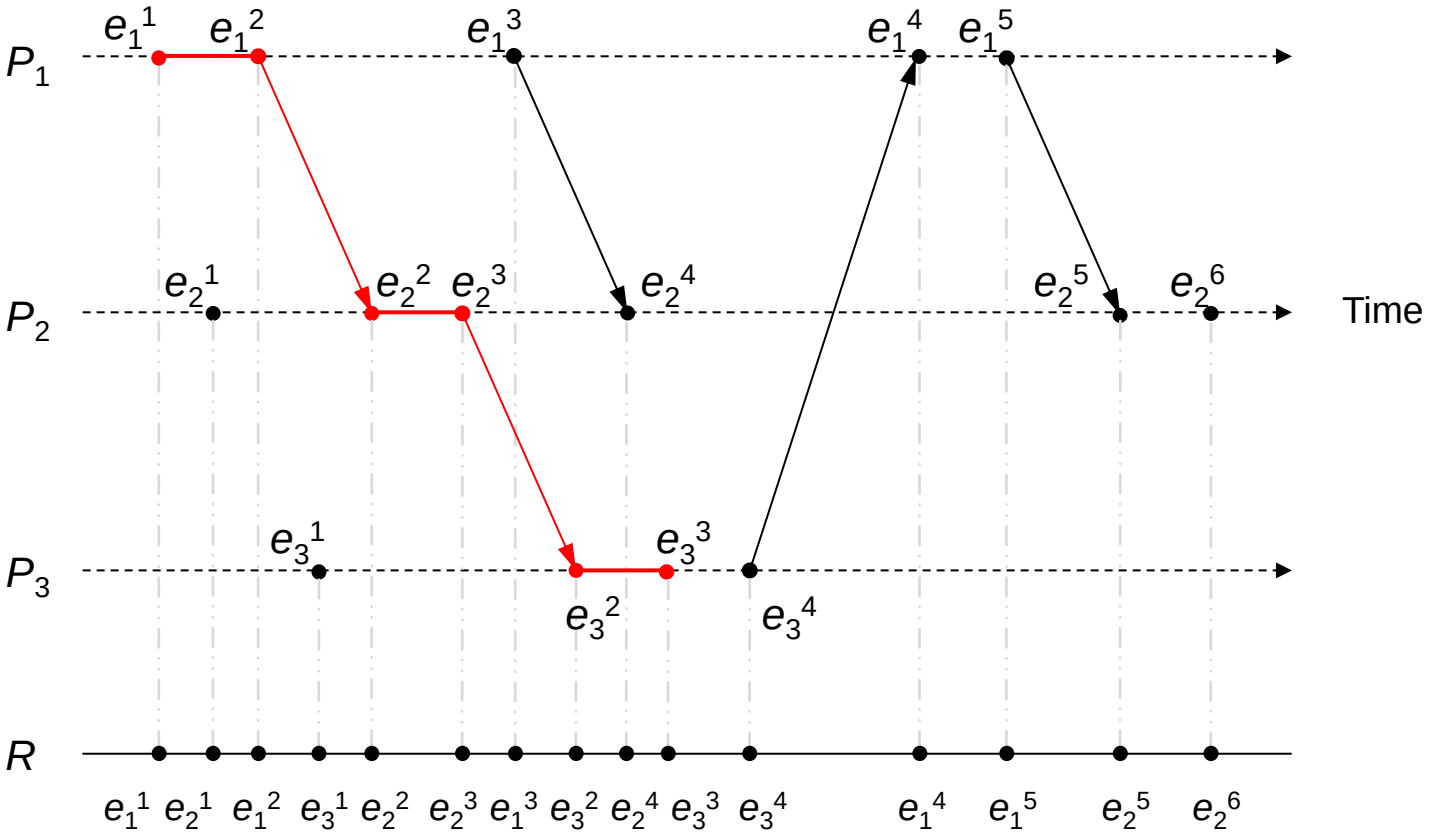
Causal order of the execution



- This relation can be extended to a partial order on the set E of events of an execution
- Namely, we say that e_i *happens before* e_j' and write $e_i \rightarrow e_j'$, if there exists a *causality chain* between these events in distributed execution, i.e.
  - if e_i and e_i' are two different events of the same process P_i and e_i occurs before e_i', then $e_i \rightarrow e_i'$
  - if e_i is a send event and e_j' the corresponding receive event, then $e_i \rightarrow e_j'$
  - $\rightarrow$ is transitive, i.e. if $e_i \rightarrow e_j'$ and $e_j' \rightarrow e_k''$, then $e_i \rightarrow e_k''$
- Causal order $\rightarrow$ was defined by Lamport

Causal order of the execution

There is a path of message arrows and process line segments starting at e_i and ending at e_j'



Causal order of the execution

- If $e_i \rightarrow e_j'$, we also say that e_i *causes* e_j' or that e_j' *knows* or *depends on* e_i
  - E.g. event e_2^6 knows about all other events on the picture
- If e_i *doesn't* cause e_j' (e_j' *doesn't* directly or transitively depend on e_i) we write $e_i \not\rightarrow e_j'$
  - E.g. event e_3^3 doesn't depend on e_1^3 in process P_1
- If an event e_j' doesn't depend on e_i then it also doesn't depend on all "future" events of P_i. Also,
  - for any two events $e_i \rightarrow e_j' \Rightarrow e_j' \not\rightarrow e_i$
  - and …… $e_i \not\rightarrow e_j' \not\Rightarrow e_j' \not\rightarrow e_i$

Causal order of the execution



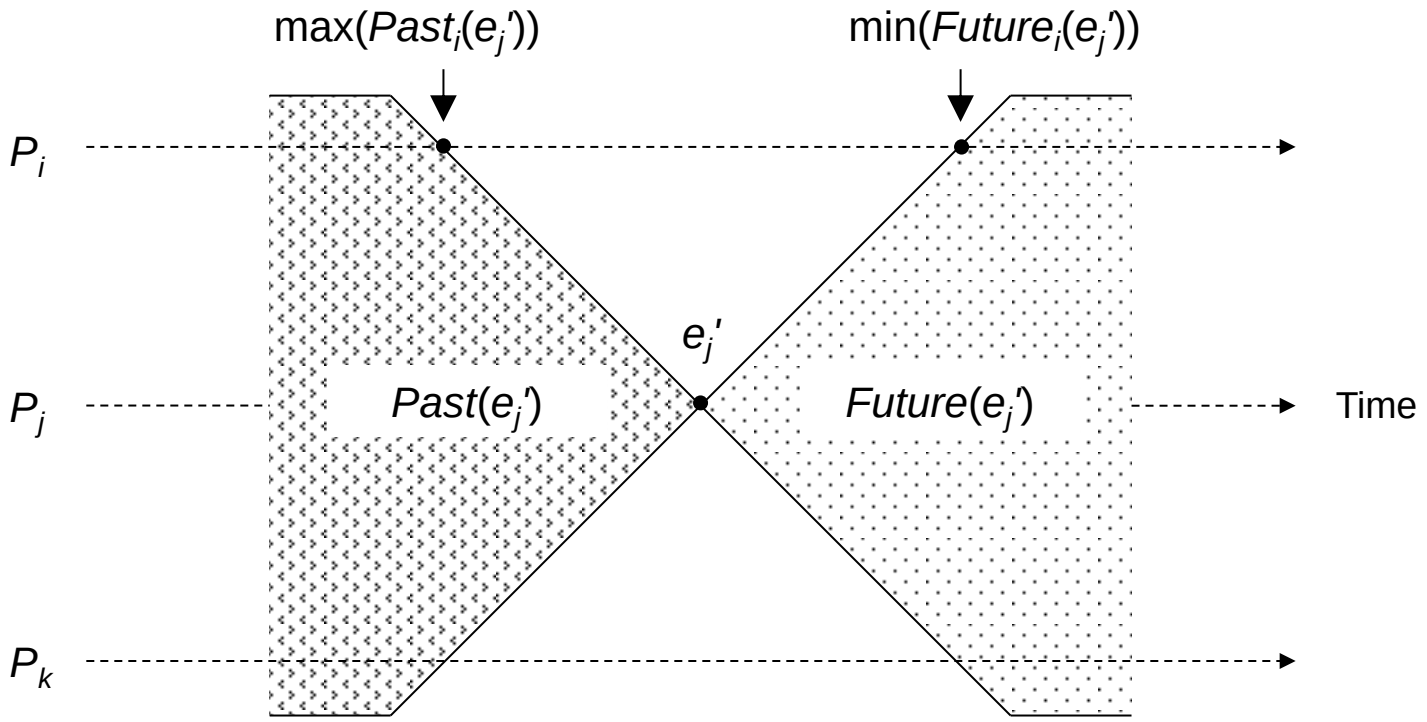
- Events e_i and e_j' such that neither $e_i \rightarrow e_j'$ nor $e_j' \rightarrow e_i$ holds are said to be *independent* or *concurrent* and we write $e_i \parallel e_j'$
  - E.g. $e_3^3 \parallel e_1^3$ on the picture
- In other words, two events are logically concurrent if and only if they do not causally affect each other
- It is important to note that relation $\parallel$ is not transitive, that is even if $e_i \parallel e_j'$ and $e_j' \parallel e_k''$ it doesn't imply that $e_i \parallel e_k''$
  - E.g. $e_3^3 \parallel e_2^4$ and $e_2^4 \parallel e_1^5$ but $e_3^3 \rightarrow e_1^5$
- Thus, for any two events e_i and e_j' in a distributed execution either $e_i \rightarrow e_j'$ or $e_j' \rightarrow e_i$ or $e_i \parallel e_j'$

Past and future cones of an event



- An event e_j' could have been caused only by all events e_i such that $e_i \rightarrow e_j'$ and all the information available at e_i could be made accessible at e_j'
- All such events e_i belong to the *past* of e_j' and we will denote such set of events as *Past*(e_j')
- The surface of the past cone of e_j' consists of the latest events at every process that caused event e_j'
- Likewise, all the events e_k that are caused by e_j' form the future of e_j', which we will denote as *Future*(e_j')
- The surface of the future cone of e_j' is defined similarly
- All events from $E \setminus Past(e_j') \setminus Future(e_j')$ are concurrent with event e_j'

Past and future cones of an event



- max($Past_i(e_j')$) is the latest event at process P_i that caused event e_j'
- min($Future_i(e_j')$) is the first event at process P_i caused by event e_j'
- What is the type of these events?

Equivalent executions



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- Now let's show that any reordering of events in execution, that preserves causal order, gives another valid execution and does not affect the result of computation
  - Though yields a different collection of global states
- Let $R = (e^0, e^1, \ldots e^x, e^{x+1}, \ldots)$ be an execution with an associated sequence of global states $(S^0, \ldots S^x, S^{x+1}, \ldots)$ where e^x *can occur* in S^x and $S^{x+1} = e^x(S^x)$ for each $x \geq 0$
- We say that the permutation $\hat{R} = (\hat{e}^0, \hat{e}^1, \ldots \hat{e}^x, \hat{e}^{x+1}, \ldots)$ of the events is *consistent* with causal order if $\hat{e}^x \rightarrow \hat{e}^y$ implies that $x < y$
  - In other words, no event $\hat{e}^y$ appears in the sequence $\hat{R}$ before an event $\hat{e}^x$ on which $\hat{e}^y$ depends

Equivalent executions



- We want to show that 1) the permutation $\hat{R}$ consistent with causal order is also an execution, i.e. $\hat{e}^x$ can occur in $\hat{S}^x$ for each $x \geq 0$, and 2) if R is finite, the last global state in $\hat{R}$ is the same as the last global state in R
- To show that $\hat{e}^x$ can occur in $\hat{S}^x$ it is necessary to show that P_i, where $\hat{e}^x$ occurs, is in proper state in $\hat{S}^x$ as well as the channel C_{ji} from where $\hat{e}^x$ receives a message (if any)
- It is important to note, that the "projection" of events from the permutation $\hat{R}$ to process P_i is *not different* from the "projection" of events from R to process P_i , i.e. the order of events at every process is *the same* in $\hat{R}$ and in R
- Thus, if $\hat{e}^x$ occurs in process P_i, then there is *the same* sequence of events in P_i preceding $\hat{e}^x$ both in $\hat{R}$ and in R

Equivalent executions



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- It means that right before occurrence of $\hat{e}^x$ in $\hat{R}$, process P_i is in *the same* state as right before occurrence of this event in execution R
- If $\hat{e}^x$ is a receive event then, it is important to note, that there is the same sequence of receive events in P_i and the same sequence of *corresponding* send events in P_j preceding $\hat{e}^x$ both in $\hat{R}$ and in R
  - Hence, the message m, to be received by $\hat{e}^x$, is in the channel C_{ji} and can be received from it
  - Does it mean that right before occurrence of $\hat{e}^x$ in $\hat{R}$ the channel C_{ji} is in the same state as right before occurrence of this event in execution R?
- Thus, each $\hat{e}^x$ can occur in $\hat{S}^x$, and $\hat{R}$ is an execution

Equivalent executions

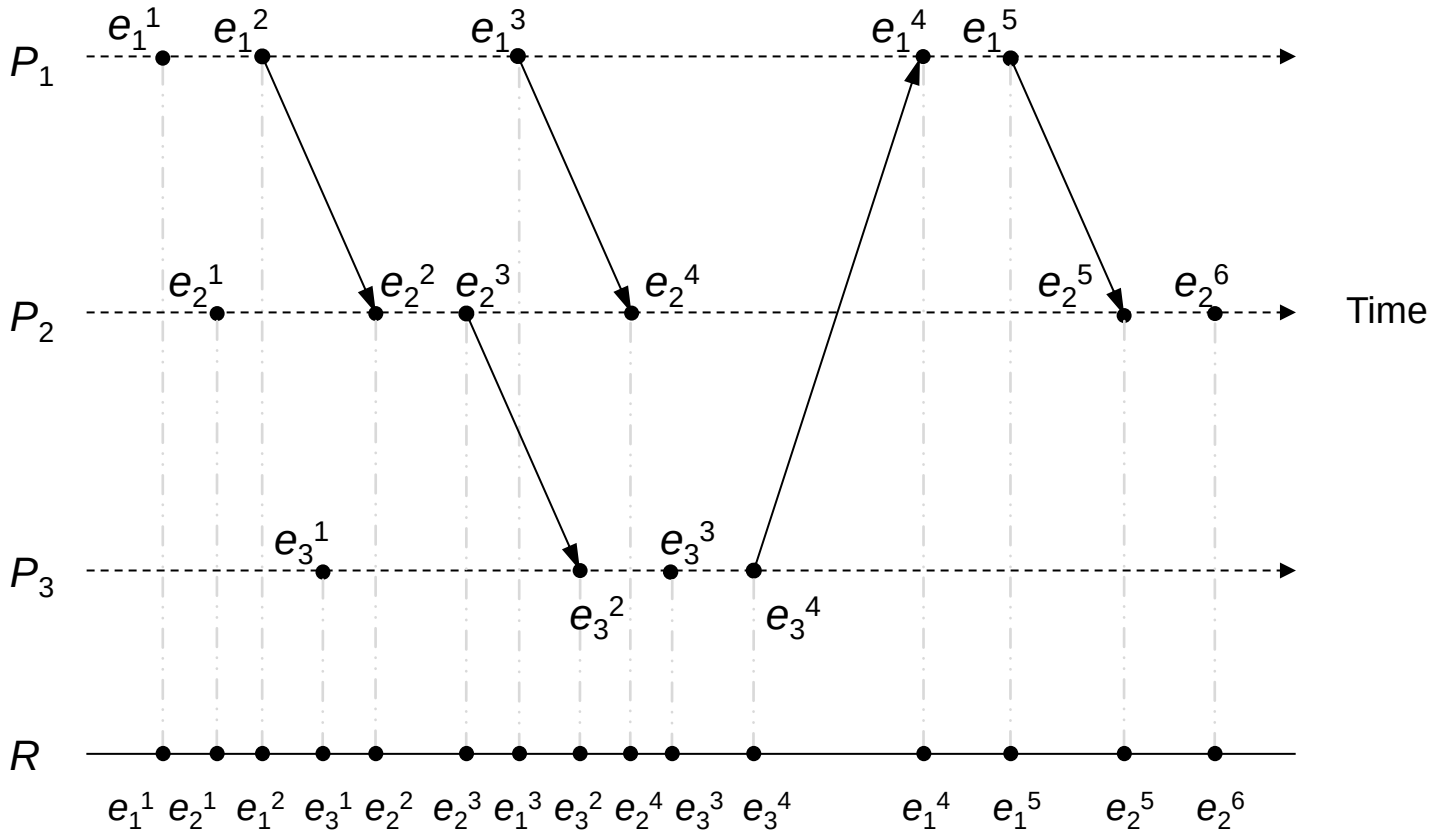


HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- Executions $\hat{R}$ and R contain the same collection of events
- The order of events at each process is the same, so if executions are finite, the last event occurring in each process and the last state of each process are the same
- Collection of send and matching receive events is the same in both executions, hence the last state of each channel (the set of messages sent but not received along each channel) is the same in $\hat{R}$ and in R
- Thus, the last global states in $\hat{R}$ and R are the same
- We will say that executions $\hat{R}$ and R containing the same collection of events with the same causal order are *equivalent* executions, denoted as $R \sim \hat{R}$

Equivalent executions

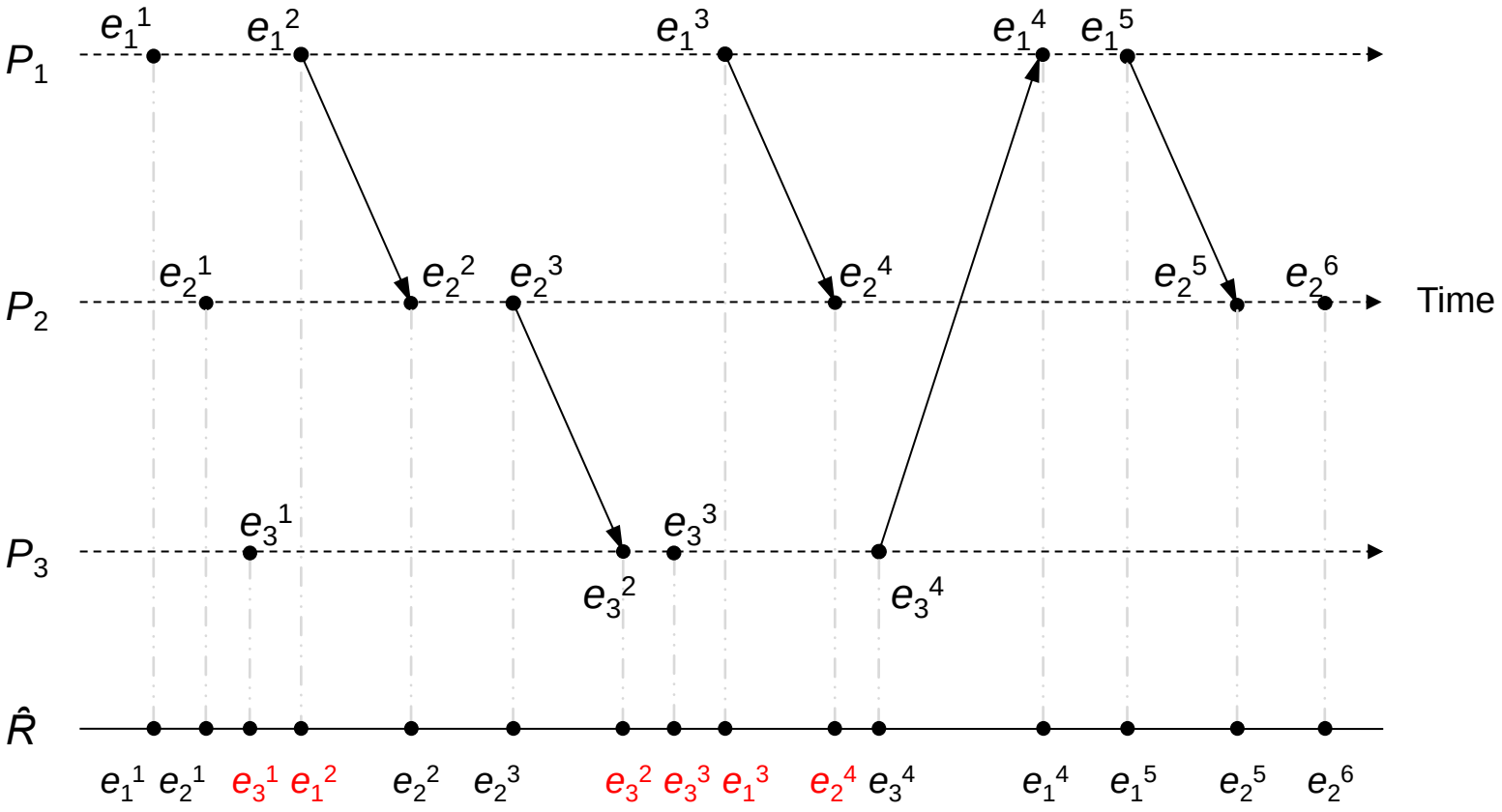
Space-time diagram of equivalent execution can be obtained by “rubber band transformations”…



Equivalent executions

HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

… time axis of a process can be compressed and stretched as long as message arrows keep pointing to the right



Equivalent executions



- Although executions depicted above are equivalent and contain the same collection of events, they do not contain the same collection of global states
- An ideal global observer, who has access to the actual sequence of events, could distinguish between two equivalent executions, i.e. could observe either one execution or another
- However, the processes can't distinguish between two equivalent executions since the same events occur in the same order at every process, and each process goes through the same sequence of states in both executions
- … i.e. equivalent executions R and $\hat{R}$ are *indistinguishable* to every process P_i